

EDA PRÁCTICA 1

Fecha: 6-X-2025

Participantes: Javier Murillo Lezaun

Javier Jon Atxutegi

Ignacio López Esteban

Índice de contenido

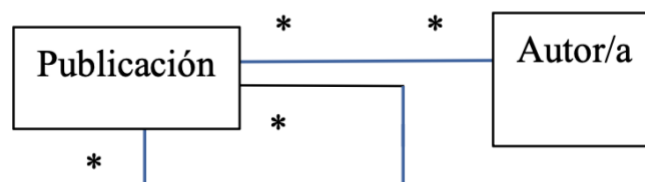
1	Introducción	1
2	Diseño de las clases.....	2
3	Descripción de las estructuras de datos principales	6
4	Diseño e implementación de los métodos principales	7
4.1	repoPublis: Método readCitas	7
4.2	repoPublis: Método readAutores.....	8
4.3	repoPublis: Método readPublicaciones	9
4.4	repoPublis: Método buscarPubliPorId	10
4.5	repoPublis: Método insertarPubli.....	10
4.6	repoPublis: Método anadirCitaAPubli	11
4.7	repoPublis: Método anadirAutorAPubli.....	11
4.8	repoPublis: Método eliminarPubli	12
4.9	repoPublis: Método listaPublisCitadas.....	12
4.10	repoPublis: Método listaAutoresPubli	13
4.11	repoPublis: Método listaPublicacionesAutor	14
4.12	repoPublis: Método ordenarAlfabeticamente.....	15
4.13	repoPublis: Método loadCitas	16
4.14	repoPublis: Método loadAutores.....	16
4.15	repoPublis: Método loadPublicaciones	17
4.16	repoAutores: Método readAutores	17
4.17	repoAutores: Método obtenerAutor	18
4.18	repoAutores: Método anadirAutor	18
4.19	repoAutores: Método eliminarAutor	19
4.20	repoAutores: Método loadAutores	19
5	Código	20
6	Conclusiones	34

1 Introducción

El problema planteado en esta actividad consiste en diseñar e implementar un sistema de gestión que permita manejar miles o incluso cientos de miles de artículos científicos, manteniendo las relaciones entre publicaciones, autores y citas de manera estructurada y accesible.

Para ello ha sido necesario implementar varias clases de dichos autores y publicaciones, relacionándolas en repositorios usando estructuras de datos desconocidas para nosotros. El principal desafío de este proyecto se encontrará en el dominio del problema, ya que una publicación puede estar escrita por varios autores y, a su vez, referenciar a otras publicaciones o ser citada por ellas. Por lo tanto, será necesario implementar algoritmos que no solo almacenen los datos dados en documentos, sino que también realizar algoritmos de búsqueda, eliminación, inserción, obtención de listas y organización.

Este proyecto representa nuestra propuesta de solución para este problema.



2 Diseño de las clases

En esta sección se mencionarán las clases principales planteadas para la solución al problema. Por cada clase, se presentarán sus características, atributos y métodos, especificando por cada uno sus propiedades (visibilidad, parámetros, tipo de los resultados, ...).



Clase Autor:

La primera clase que implementaremos será la clase Pública Autor, ya que esta será atributo de la clase Publicación. Los atributos de esta clase serán:

- Un String con el identificador del autor (pId) (Privado)
- Un String con el nombre del autor (pAutor) (Privado)

Los métodos de esta clase serán principalmente métodos Setters y Getters para poder utilizarlos en futuras clases y métodos. Estos son:

- `public Autor (String pId, String pNombre):` Método constructor de la clase Autor
- `public String getNombre():` Método Getter para obtener el nombre del Autor en forma de String.
- `public void setNombre(String nombre):` Método Setter para establecer el String que recibe como parámetro como nombre del Autor.
- `public String getId():` Método getter del identificador del Autor.
- `public void setId(String id):` Método Setter para establecer el String que recibe como parámetro como identificador del Autor.

Clase Publicación:

La siguiente clase que implementamos es la clase Pública Publicación, la cual, similarmente a la anterior, la utilizaremos para crear un repositorio de este tipo. Los atributos de esta clase serían:

- Un String con el identificador de la publicación (pId) (Privado)
- Un String con el título de la Publicación (pNombre) (Privado)

Los métodos de esta clase, similar a la anterior, serán métodos Getters para poder acceder a sus atributos en futuras clases:

- public Publicación(String pId, String pNombre): El método constructor de la clase Publicación.
- public String getId(): Un método Getter para acceder al Identificador de la publicación.
- public String getTitulo(): Un método Getter para acceder al título de la publicación.

Clase RepositorioAutores:

La clase pública RepositorioAutores será la clase en la que iniciaremos un HashMap para guardar en ella a todos los autores con sus respectivos identificadores. Para ello su único atributo será un HashMap privado con dos atributos, un String (Identificador del autor) y un objeto autor de la clase Autor.

En cuanto a los métodos tenemos los siguientes:

- public void insertarAutor(String pId, String pNom) : Método tipo void que añade un autor al repositorio dados su Identificador y Nombre.
- public void readAutores(String nom) : Este método inicia un Scanner que dado un documento con información consigue extraer de él los identificadores y Nombres de cada autor. En este caso cada línea de documento separa los datos por una almohadilla (#), por lo que el método guarda la parte izquierda a la almohadilla el Id y la derecha en el nombre del autor.
- public Autor obtenerAutor(String pId): este método, dado un identificador en forma de String, devuelve el autor del repositorio al que le corresponde el Id.
- public void eliminarAutor(String pId): este método, dado un identificador en forma de String, elimina el autor del repositorio al que le corresponde el Id.
- public void loadAutores(String nom): Este método por cada autor que tenga el repositorio, actualiza en el fichero cada uno con su nombre e identificador, separados por una almohadilla.
- public HashMap<String, Autor> getRepositorioAutores(): Este método devuelve la instancia del repositorio de autores.

Clase RepositorioPublicaciones:

La clase pública RepositorioPublicaciones consiste en un repositorio con todas las publicaciones, para el cual tenemos como atributos para él tres Hashmaps:

- Hashmap de las citas: Contiene un ArrayList de Strings en la que están guardadas todas las citas y un Identificador en forma de String para cada ArrayList.
- Hashmap de las publicaciones: Contiene todas las publicaciones y su identificador correspondiente a cada una de ellas.
- Hashmap de autores: Este hasmap contiene un ArrayList de todos los autores correspondientes a la publicación y el identificador de cada publicación.

En cuanto a los métodos, tenemos:

- `public void readCitas(String nom)`: Método void cuya funcionalidad es leer todas las entradas del documento, guardando por cada línea el identificador de la publicación correspondiente y la cita, añadiéndola en el ArrayList de dicha publicación con su Identificador.
- `public void readAutores(String nom)`: Similar al anterior, este método lee cada entrada del documento y guarda por cada línea en el Hashmap de autores, los String correspondientes a los autores de la publicación.
- `public void readPublicaciones(String nom)`: En este método se guarda por cada línea del documento la publicación y su respectivo identificador.
- `public Publicacion buscarPubliPorId(String pId)`: Dado un identificador, el método busca en el Hashmap de publicaciones la publicación con el respectivo identificador.
- `public void insertarPubli(String pId, String pTítulo)`: Dado un título en forma de String y un identificador, el método añade en el Hashmap de publicaciones esa publicación con el respectivo identificador.
- `public void anadirCitaAPubli(String pId, String pCita)`: Este método inserta una cita al Hashmap de citas, añadiendo la cita en forma de String en la publicación correspondiente al identificador dado. Si en el hashmap no se encuentra el identificador dado, este crea una nueva entrada con dicho identificador.
- `public void anadirAutorAPubli(String pIdPubli, String pIdAutor)`: Similar al anterior, este método inserta un autor en forma de Strings al Hashmap de autores, añadiendo el autor en forma de String en la publicación correspondiente al identificador dado. Si en el hashmap no se encuentra el identificador dado, este crea una nueva entrada con dicho identificador.
- `public void eliminarPubli(String pIdPubli)`: Este método dado un Identificador en forma de String, elimina la publicación del Hashmap de publicaciones con dicho Identificador.
- `public ArrayList<Publicacion> listaPublisCitadas(String pId)`: Este método devuelve una lista con todas las publicaciones que hayan estado citadas en la lista correspondiente al identificador que recibe.
- `public ArrayList<String> listaAutoresPubli(Publicacion p)`: Este método recibe una publicación y devuelve una lista con todos los autores que hayan participado en esta.
- `public ArrayList<Publicacion> listaPublicacionesAutor(Autor a)`: Este método, dado un autor, devuelve todas las publicaciones en las cuales dicho autor haya trabajado, es decir, todas las publicaciones en las que ese autor se encuentre en el Hashmap de publicaciones.
- `public ArrayList<Publicacion> ordenarAlfabeticamente()`: Este método devuelve una lista de tipo ArrayList de publicaciones previamente ordenada alfabéticamente.
- `public void loadCitas(String nom)`: Este método void que recibe un nombre en forma de String actualiza en el fichero cada cita por publicación que haya en el Hashmap de citas, siendo separadas por una almohadilla.
- `public void loadAutores(String nom)`: Este método void que recibe un nombre en forma de String actualiza en el fichero cada autor por publicación que haya en el Hashmap de autores, siendo separadas por una almohadilla.
- `public void loadPublicaciones(String nom)`: Este método void que recibe un nombre en forma de String actualiza en el fichero cada publicación que haya en el Hashmap de citas, siendo separadas por una almohadilla.

- `public HashMap<String, Publicación> getPublicaciones():` Este método devuelve la instancia del repositorio de publicaciones.
- `public HashMap<String, ArrayList<String>> getCitas():` Este método devuelve la instancia del repositorio de citas.
- `public HashMap<String, ArrayList<String>> getAutores():` Este método devuelve la instancia del repositorio de autores

Clase Main:

La última clase que implementamos es la clase Main. Será la encargada de ejecutar los métodos y calcular cuantos milisegundos/segundos tarda en ejecutarse cada uno de los métodos principales. y no triviales, es decir, aquellos que requieran recorrer varios HashMap o ArrayList. Sus únicas variables locales son:

- Un Repositorio de Autores.
- Un Repositorio de Publicaciones.
- Los milisegundos del Inicio y los milisegundos del Final.

En cuanto a los métodos tenemos los siguientes:

- `public static main(String[] args) :` Método estático principal del programa, que se encargará de ejecutar todos los métodos principales del programa, a la vez que calcula los tiempos de ejecución de estos y los muestra por consola.

3 Descripción de las estructuras de datos principales

Este apartado describirá las principales estructuras diseñadas para la solución del problema, detallando sus características principales, así como la justificación de su idoneidad para resolver el problema planteado.

Para la solución del problema que se nos ha planteado hemos optado por utilizar la estructura HashMap como base para organizar toda la información que se nos ha proporcionado.

El motivo principal que nos ha llevado a la elección final es la eficiencia y comodidad que nos proporcionaría el HashMap a la hora de relacionar los distintos elementos y datos que se nos presentan mediante una clave única, que en nuestro caso se trataba del ID de la publicación en el caso del Repositorio de Publicaciones y del ID del autor en el caso del Repositorio de Autores. De esta forma, evitábamos tener que recorrer listas de objetos o acceder a los atributos de las diferentes clases manualmente para encontrar coincidencias, ya que el HashMap nos permitía obtener en tiempo constante $O(1)$ la información asociada a una clave.

Concretamente hemos definido cuatro HashMaps:

- publicaciones (en repositorio publicaciones): asocia cada ID de publicación con su objeto Publicación.
- citas (en repositorio publicaciones): asocia cada ID de publicación con una lista (`ArrayList<String>`) con los IDs de las publicaciones que se citan en dicha publicación.
- autores (en repositorio publicaciones): asocia cada ID de publicación con una lista (`ArrayList<String>`) con los IDs de los autores que participan en dicha publicación.
- autores (en repositorio autores): asocia cada ID de autor con su objeto Autor.

Hemos decidido combinar los HashMaps con los ArrayList para mantener en el HashMap una búsqueda directa y fácilmente accesible de una publicación o de sus datos asociados. Manteniendo además de manera ordenada dichos datos almacenados mediante los ArrayList que guardan varias instancias de un mismo dato, las varias citas que puede tener una publicación o los varios autores que puede tener una publicación.

Gracias a esta combinación de tipos de datos podemos lograr acceder de una manera eficiente a la información, mantener todos los datos relacionados, publicaciones, citas y autores sin ninguna incongruencia, y a su vez simplificar el diseño de la solución.

4 Diseño e implementación de los métodos principales

En este apartado se presentarán los métodos principales, indicando por cada uno su especificación (precondición y postcondición), además de los casos de prueba que hemos planteado. Por cada método no trivial (no han sido incluidos los getters, cuyos costes son $O(1)$), se presentará la descripción del algoritmo implementado. Finalmente, se presentará, de manera razonada, el cálculo del coste de cada algoritmo, indicando a que tasa de crecimiento pertenece.

4.1 Clase *repoPublis*: Método *readCitas*

```
public void readCitas(String nom) {  
    /* Precondición: El parámetro nom no es null ni cadena vacía.  
    /* Postcondición: Los archivos se habrán cargado desde los  
    ficheros.
```

Casos de prueba:

- Comprobar que el tamaño del repositorio creado es distinto de 0.
- Comprobar que el repositorio contiene el primer Id del fichero.
- Comprobar que el primer Id tiene asociada su respectiva cita.

Implementación del algoritmo:

INICIO

 INTENTAR

 abrir archivo con NombreArchivo para lectura

 Mientras haya más líneas en el archivo HACER

 leer línea

 datos[] := separar datos usando " # " como delimitador

 datos[0] ← Id

 datos[1] ← Id publicación citada.

 si las citas no contienen el Id de la pos0 ENTONCES

 añadir ese Id y una lista vacía a las citas

 FIN SI;

 Añadir la cita a la lista de esa publicación

 FIN MIENTRAS;

 Cerrar archivo;

 Si ocurre un error de entrada/salida

 Mostrar error

 FIN INTENTAR;

FIN

Coste: El algoritmo examina todas las líneas del fichero. Para un fichero de n líneas, el coste será $O(n)$.

4.2 Clase *repoPublis*: Método *readAutores*

```
public void readAutores(String nom) {  
    /* Precondición: El parámetro nom no es null ni cadena vacía.  
    /* Postcondición: Los archivos se habrán cargado desde los  
    ficheros.
```

Casos de prueba:

- Comprobar que el tamaño del repositorio creado es distinto de 0.
- Comprobar que el repositorio contiene el primer Id del fichero.
- Comprobar que el primer Id tiene asociado su autor (Id)

Implementación del algoritmo:

```
INTENTAR  
    abrir archivo con NombreArchivo para lectura  
    mientras haya líneas que leer en el archivo HACER  
        leer línea  
        datos[] := separar datos usando " # " como delimitador  
        datos[0] ← Id Publicación  
        si los autores no contienen el Id de la pos0 ENTONCES  
            añadir ese Id y una lista vacía al mapa autores.  
        FIN SI;  
        Si la longitud de datos es 2 ENTONCES  
            datos[1] ← Id del autor que la ha realizado  
            se guarda en la lista de la publicación el id  
            del autor.  
        FIN SI;  
    FIN MIENTRAS;  
    Cerrar archivo;  
    Si ocurre un error de entrada/salida  
        Mostrar error  
    FIN INTENTAR;  
FIN
```

Coste: El algoritmo examina todas las líneas del fichero. Para un fichero de n líneas, el coste será $O(n)$.

4.3 Clase *repoPublis*: Método *readPublicaciones*

```
public void readPublicaciones(String nom) {  
    /* Precondición: El parámetro nom no es null ni cadena vacía.  
    /* Postcondición: Los archivos se habrán cargado desde los  
    ficheros.
```

Casos de prueba:

- Comprobar que el tamaño del repositorio creado es distinto de 0.
- Comprobar que el repositorio contiene el primer Id del fichero.
- Comprobar que el primer Id tiene asociado su título.

Implementación del algoritmo:

INICIO

 INTENTAR

 abrir archivo con NombreArchivo para lectura

 mientras haya líneas que leer en el archivo HACER

 leer línea

 datos[] := separar datos usando " # " como delimitador

 datos[0] ← Id publicación

 datos[1] ← Título publicación

 Se crea una publicación con su nombre y su título.

 Se añade esa publicación al mapa de publicaciones.

 FIN MIENTRAS;

 Cerrar archivo;

 Si ocurre un error de entrada/salida

 Mostrar error

 FIN INTENTAR;

FIN

Coste: El algoritmo examina todas las líneas del fichero. Para un fichero de n líneas, el coste será $O(n)$.

4.4 Clase *repoPublis*: Método *buscarPubliPorId*

```
public Publicación buscarPubliPorId(String pId) {  
    /* Precondición: El parámetro pId no es null ni cadena vacía.  
    /* Postcondición: Devolverá el objeto publicación asociado a  
    ese Id o null en caso de no existir.
```

Casos de prueba:

- Búsqueda de un Id existente en el HashMap.
 - La primera y última publicación
- Búsqueda de un Id no existente en el HashMap.

Implementación del algoritmo:

INICIO

```
    Publi ← mapapublicaciones[pId]; //Guarda el valor asociado a la  
    clave pId  
    Return Publi;
```

FIN

Coste: El algoritmo accede directamente al valor asociado a la clave pId del HashMap , el coste será $O(1)$.

4.5 Clase *repoPublis*: Método *insertarPubli*

```
public void insertarPubli(String pId, String pTítulo) {  
    /* Precondición: El parámetro pId y pTítulo no es null ni  
    cadena vacía.  
    /* Postcondición: El HashMap de las publicaciones tendrá una  
    nueva publicación con clave pId, y como valor un objeto de  
    Publicación con identificador pId y título pTítulo.
```

Casos de prueba:

- Insertar una publicación nueva.
 - Comprobar que el tamaño de antes de añadir incrementa en una unidad tras añadirla.

Implementación del algoritmo:

INICIO

```
    p ← Crear un nuevo objeto Publicación con pId y pTítulo.  
    Insertarlo en el mapa de publicaciones con clave pId y objeto  
    p como valor.
```

FIN

Coste: El algoritmo accede directamente al HashMap e inserta la publicación en una sola instrucción, como la inserción tiene tiempo constante, el coste será $O(1)$.

4.6 Clase *repoPublis*: Método *anadirCitaAPubli*

```
public void anadirCitaAPubli(String pId, String pTítulo) {  
    /* Precondición: El parámetro pId y pTítulo no es null ni  
    cadena vacía.  
    /* Postcondición: El HashMap de citas tendrá una nueva  
    publicación en la lista de citas cuya clave es el valor pId si  
    la publicación pId existe en el HashMap.
```

Casos de prueba:

- Insertar una cita a una publicación que ya tiene varias citas.
- Insertar una cita a una publicación cuya clave existe en el HashMap, pero no tiene citas.

Implementación del algoritmo:

INICIO

```
    Si el mapa no contiene el valor de pId ENTONCES  
        Crear una lista vacía  
        Asociar esa lista al valor pId en el mapa de citas  
    FIN SI;  
    Obtener la lista de citas asociada a pId  
    Añadir pCita a esa lista.
```

FIN

Coste: El algoritmo accede directamente a los valores del HashMap y los añade, y como dichas operaciones son $O(1)$ en promedio, el coste será $O(1)$.

4.7 Clase *repoPublis*: Método *anadirAutorAPubli*

```
public void anadirAutorAPubli(String pIdPubli, String pNom) {  
    /* Precondición: El parámetro pId y pCita no es null ni cadena  
    vacía.  
    /* Postcondición: El mapa de autores tendrá un nuevo autor en  
    la lista de autores cuya clave es el valor pId si la publicación  
    pId existe en el HashMap.
```

Casos de prueba:

- Insertar un autor a una publicación que ya tiene varios autores.
- Insertar una cita a una publicación cuya clave existe en el HashMap, pero no tiene autores.

Implementación del algoritmo:

INICIO

```
    Si el mapa no contiene el valor de pId ENTONCES  
        Crear una lista vacía  
        Asociar esa lista al valor pId en el mapa de autores  
    FIN SI;  
    Obtener la lista de autores asociada a pId  
    Añadir pNom a esa lista.
```

FIN

Coste: El algoritmo accede directamente a los valores del HashMap, y como las instrucciones son condicionales e inserciones con tiempo constante, el coste será $O(1)$.

4.8 Clase *repoPublis*: Método *eliminarPubli*

```
public void eliminarPubli(String pIdPubli) {  
    /* Precondición: El parámetro pIdPubli no es null ni cadena vacía.  
    /* Postcondición: El HashMap de publicaciones no contendrá ni la clave pIdPubli ni el valor asociado a esa clave.
```

Casos de prueba:

- Eliminar una publicación cuyo Id existe en el HashMap.
- Eliminar una publicación cuyo Id no existe en el HashMap.

Implementación del algoritmo:

INICIO

```
    Eliminar la clave pId y su valor asociado del mapa de publicaciones.
```

FIN

Coste: El algoritmo accede directamente al valor del HashMap y lo elimina, por lo que el coste será $O(1)$.

4.9 Clase *repoPublis*: Método *listaPublisCitadas*

```
public ArrayList<Publicación> listaPublisCitadas(String pId){  
    /* Precondición: El parámetro pId no es null ni cadena vacía.  
    /* Postcondición: Devuelve una lista con todas las publicaciones citadas de esa publicación si existe la clave.
```

Casos de prueba:

- Utilizar como clave una que no existe en el HashMap.
- Utilizar como clave una que sí existe en el HashMap.
 - Se comprueba que la lista devuelta por el método tiene el mismo tamaño que la lista asociada a ese Id.

Implementación del algoritmo:

INICIO

```
    lPC<-Lista vacía con objetos Publicación.  
    Lista<-citas[pId] //Citas es el mapa asociado a la clave pId.  
    Si la lista no tiene valor null ENTONCES  
        Para cada id en la lista HACER  
            Publi<-publicaciones[id] //Publicaciones es el mapa asociado al id.  
            Añadir publi a lPC  
        FIN PARA;  
    Return lPC
```

FIN

Coste: El algoritmo examina todos los elementos de la lista de citas. Para una lista de n citas, el coste será $O(n)$.

4.10 Clase *repoPublis*: Método *listaAutoresPubli*

```
public ArrayList<String> listaAutoresPubli(Publicación p){  
    /* Precondición: El parámetro p no es null.  
    /* Postcondición: Devuelve una lista con todos los autores que  
    han hecho una determinada publicación.
```

Casos de prueba:

- Utilizar como clave una publicación que sí existe en el HashMap.
- Se comprueba que la lista devuelta por el método tenga el mismo tamaño que la lista de autores.
- Se comprueba que ambas listas contienen una misma publicación
- Utilizar como clave una publicación que no existe en el HashMap.

Implementación del algoritmo:

INICIO

```
    Se consigue el pId de la publicación p  
    Lista ← mapaAutores[pId] //se accede al mapa y devuelve la lista  
    de autores  
    Return Lista;
```

FIN

Coste: El algoritmo accede directamente al valor del HashMap y lo devuelve, única instrucción, por lo que el coste será $O(1)$.

4.11 Clase *repoPublis*: Método *listaPublicacionesAutor*

```
Public ArrayList<Publicación> listaPublicacionesAutor (Autor
a){
/* Precondición: El parámetro a no es null.
/* Postcondición: Devuelve una lista con todas las
publicaciones que ha hecho el autor pasado por parámetro.
```

Casos de prueba:

- Utilizar como clave una publicación que no existe en el HashMap.
- Se comprueba que el tamaño de la lista obtenida es 0.
- Utilizar como clave una que sí existe en el HashMap.
- Se comprueba que la lista obtenida contiene la publicación pasada por parámetro.

Implementación del algoritmo:

INICIO

Se crea una lista vacía

PARA CADA idPubli en el mapa de autores REPETIR

Si la lista asociada idPubli es distinta de null ENTONCES

$lA \leftarrow \text{mapaAutores}[\text{idPubli}]$

 si el mapa contiene el id del autor pasado por parámetro ENTONCES

 se accede al mapa de publicaciones y se guarda en la lista el valor de la Publicación cuyo id es idPubli.

 FIN SI;

FIN SI;

FIN PARA;

FIN

Coste: El algoritmo examina todos los ids del mapa de autores. Para un HashMap de n ids, el coste será $O(n)$.

4.12 Clase *repoPublis*: Método *ordenarAlfabeticamente*

```
Public ordenarAlfabeticamente() {  
    /* Precondición:  
    /* Postcondición: Devuelve una lista ordenada de publicaciones
```

Casos de prueba:

- Comprobar que el último elemento es mayor que el primero.

Implementación del algoritmo:

INICIO

```
    Se crea una lista vacía  
    Para cada publicación publi en el mapa de publicaciones REPETIR  
        Si el tamaño de lista es == 0 ENTONCES  
            Lista ← añadir esa publicación publi  
        Si no  
            posInicial ← 0;  
            posFinal ← longitud de lista - 1;  
            posMitad ← 0  
            enc ← false;  
            mientras (posInicial ≤ posFinal y no se haya  
                encontrado) REPETIR  
                posMitad ← (posInicial + posFinal) / 2;  
                se obtiene la publicación p en esa posMitad;  
                si el ID de la p de la mitad es menor que el de  
                    publi ENTONCES  
                    posInicial ← posInicial + 1; (la pos en donde  
                        se insertará está a la derecha)  
                si no  
                    si la longitud de la lista es 0 o la  
                        publicación de la pos anterior es menor  
                            ENTONCES  
                                enc ← true; (se ha encontrado la pos)  
                    si no  
                        posInicial ← posInicial - 1; (la pos en  
                            donde se insertará está a la izq)  
                        FIN SI;  
                    FIN SI;  
            FIN MIENTRAS;  
            Si enc == False (no se ha encontrado la pos) ENTONCES  
                se añade al final de la lista;  
            si no  
                se añade en la posición de la mitad.  
            FIN SI;  
        FIN PARA;  
    Return lista;
```

FIN

Coste: El algoritmo examina todos los elementos de la lista, donde n se corresponde al número de elementos de la lista y m es el número de veces que entra al while. Divide constantemente la lista en 2, por lo que el coste será $O(n \cdot \log m)$.

4.13 Clase *repoPublis*: Método *loadCitas*

```
Public void loadCitas(String nom){  
    /* Precondición: El parámetro nom no es null ni cadena vacía.  
    /* Postcondición: Los datos se actualizarán en el fichero.
```

Casos de prueba:

- Crear un archivo nuevo y comprobar que tras realizar cambios se guardan las actualizaciones.

Implementación del algoritmo:

INICIO

 INTENTAR

 abrir archivo con NombreArchivo para lectura

 Para cada pId en el mapa de citas REPETIR

 Para cada pCita en la lista de citas REPETIR

 Escribir en el archivo pId+" # "+pCita

 FIN PARA;

 FIN PARA;

 Guardar cambios en el archivo;

 Cerrar archivo;

 Si ocurre un error de entrada/salida

 Mostrar error

 FIN INTENTAR;

FIN

Coste: El algoritmo examina todas las citas del HashMap. Para un HashMap de n citas el coste será $O(n)$.

4.14 Clase *repoPublis*: Método *loadAutores*

```
Public void loadAutores(String nom){  
    /* Precondición: El parámetro a no es null.  
    /* Postcondición: Los datos se actualizarán en el fichero.
```

Casos de prueba:

- Crear un archivo nuevo y comprobar que tras realizar cambios se guardan las actualizaciones.

Implementación del algoritmo:

INICIO

 INTENTAR

 abrir archivo con NombreArchivo para lectura

 Para cada pId en el mapa de Autores REPETIR

 Para cada pAutor en la lista de citas REPETIR

 Escribir en el archivo pId+" # "+pAutor

 FIN PARA;

 FIN PARA;

 Guardar cambios en el archivo;

 Cerrar archivo;

 Si ocurre un error de entrada/salida

 Mostrar error

 FIN INTENTAR;

FIN

Coste: El algoritmo examina todos los autores del HashMap. Para un HashMap de n autores será $O(n)$.

4.15 Clase *repoPublis*: Método *loadPublicaciones*

```
Public void loadPublicaciones(String nom){  
/* Precondición: El parámetro a no es null.  
/* Postcondición: Los datos se actualizarán en el fichero.
```

Casos de prueba:

- Crear un archivo nuevo y comprobar que tras realizar cambios se guardan las actualizaciones.

Implementación del algoritmo:

```
INICIO  
  INTENTAR  
    abrir archivo con NombreArchivo para lectura  
    Para cada pId en el mapa de Autores REPETIR  
      Escribir en el archivo pId+" # "+Título publi  
    FIN PARA;  
    Guardar cambios en el archivo;  
    Cerrar archivo;  
  Si ocurre un error de entrada/salida  
    Mostrar error  
  FIN INTENTAR;  
FIN
```

Coste: El algoritmo examina todas las publis del HashMap. Para un HashMap de n publis será $O(n)$.

4.16 Clase *repoAutores*: Método *readAutores*

```
Public void readAutores(String nom){  
/* Precondición: El parámetro nom no es null ni cadena vacía.  
/* Postcondición:
```

Casos de prueba:

- Comprobar que el método que devuelve la instancia de autores no devuelve null.
- Comprobar que un determinado título se corresponde al valor asociado a su Id.

Implementación del algoritmo:

```
INICIO  
  INTENTAR  
    abrir archivo con NombreArchivo para lectura  
    mientras haya líneas que leer en el archivo HACER  
      leer línea  
      datos[] := separar datos usando " # " como delimitador  
      datos[0] ← Id del Autor;  
      datos[1] ← Nombre Autor;  
      Se crea un objeto autor con esos dos datos.  
      añadir ese objeto autor al mapa de autores.  
    FIN MIENTRAS;  
    Cerrar archivo;  
  Si ocurre un error de entrada/salida  
    Mostrar error  
  FIN INTENTAR;  
FIN
```

Coste: El algoritmo examina todas las líneas del fichero. Para un fichero de n líneas será $O(n)$.

4.17 Clase *repoAutores*: Método *obtenerAutor*

```
Public Autor readAutores(String pId){  
/* Precondición: El parámetro nom no es null ni cadena vacía.  
/* Postcondición: Devuelve el objeto Autor asociado a la clave  
pId
```

Casos de prueba:

- Comprobar que, tras añadir un nuevo autor, el Autor que el método devuelve:
 - Su id se corresponde al del autor añadido.
 - Su Nombre se corresponde al del autor añadido.
- Comprobar que el autor que devuelve no es null.

Implementación del algoritmo:

```
INICIO  
    Return el autor asociado a la clave pId;  
FIN
```

Coste: El algoritmo accede directamente al valor del HashMap, por lo que el coste será $O(1)$.

4.18 Clase *repoAutores*: Método *anadirAutor*

```
Public void anadirAutor(String pId,String pNom){  
/* Precondición: Los parámetros pId y pNom no son null ni  
cadena vacía.  
/* Postcondición: El HashMap de autores tendrá uno más.
```

Casos de prueba:

- Comprobar que, tras añadir un nuevo autor, el Autor que el método devuelve:
 - Su id se corresponde al del autor añadido.
 - Su Nombre se corresponde al del autor añadido.
- Comprobar que el autor que devuelve no es null.

Implementación del algoritmo:

```
INICIO  
    Se crea un objeto autor con pId y pNom;  
    Se añade al mapa de autores, cuya clave será pId.  
FIN
```

Coste: El algoritmo accede directamente al valor del HashMap, y como la operación es una inserción, el coste será $O(1)$.

4.19 Clase repoAutores: Método eliminarAutor

```
Public void eliminarAutor(String pId){  
    /* Precondición: El parámetro pId no es null ni cadena vacía.  
    /* Postcondición:
```

Casos de prueba:

- Comprobar que, tras añadir un nuevo autor, el método:
 - Guardar el tamaño y tras eliminar el autor comprobar que no coinciden los tamaños.
 - Comprobar que el repositorio de autores NO contiene a ese autor.

Implementación del algoritmo:

```
INICIO  
    Eliminar el autor asociado a la clave pId;  
FIN
```

Coste: El algoritmo accede directamente al valor del HashMap y lo elimina, por lo que el coste será $O(1)$.

4.20 Clase repoAutores: Método loadAutores

```
Public void loadAutores(String nom){  
    /* Precondición: El parámetro nom no es null ni cadena vacía.  
    /* Postcondición:
```

Casos de prueba:

- Crear un archivo nuevo y comprobar que tras realizar cambios se guardan las actualizaciones.

Implementación del algoritmo:

```
INICIO  
I    INTENTAR  
    abrir archivo con NombreArchivo para lectura  
    Para cada autor a en el mapa de Autores REPETIR  
        Escribir en el archivo id autor+" # "+nombre autor  
    FIN PARA;  
    Guardar cambios en el archivo;  
    Cerrar archivo;  
    Si ocurre un error de entrada/salida  
        Mostrar error  
FIN INTENTAR;  
FIN
```

Coste: El algoritmo examina todas las líneas del HashMap de autores. Para un HashMap de n autores, el coste será $O(n)$.

5 Código

En este apartado se presentará el código de las clases, junto con los programas de prueba o Junits desarrollados.

Se puede insertar un índice eligiendo “Insertar” → “Índices” → “Índices” y después la pestaña “Índice” (opción “índice de contenido”).

1. [Autor](#)
2. [Publicación](#)
3. [RepositorioAutores](#)
4. [RepositorioPublicaciones](#)
5. [Main](#)

AUTOR

[volver](#)

```
public class Autor {  
  
    private String id;  
    private String nombre;  
  
    public Autor(String pId, String pNombre) {  
        this.id = pId;  
        this.nombre = pNombre;  
    }  
  
    public String getId() {  
        return id;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

PUBLICACION

volver

```
public class Publicacion {  
  
    private String id;  
    private String titulo;  
  
    public Publicacion(String pId, String pNombre) {  
        this.id = pId;  
        this.titulo = pNombre;  
    }  
  
    public String getId() {  
        return id;  
    }  
  
    public String getTitulo() {  
        return titulo;  
    }  
}
```

REPOSITORIO AUTORES

volver

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.HashMap;
import java.util.Scanner;

public class RepositorioAutores {
    private HashMap<String, Autor> autores; //ID autor # Objeto Autor

    public RepositorioAutores() {
        autores = new HashMap<>();
    }

    public void anadirAutor(String pId, String pNom) {
        Autor a = new Autor(pId,pNom);
        this.autores.put(pId, a);
    }

    public void readAutores(String nom) {
        try {
            Scanner entrada = new Scanner(new FileReader(nom));
            String linea = null;
            while(entrada.hasNextLine()) {
                linea = entrada.nextLine();
                String[] datos = linea.split(" # ");
                Autor a = new Autor(datos[0],datos[1]);
                autores.put(datos[0], a);
            }
            entrada.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

    public Autor obtenerAutor(String pId) {
        return autores.get(pId);
    }

    public void eliminarAutor(String pId) {
        autores.remove(pId);
    }

    public void loadAutores(String nom) {
        try {
            PrintWriter salida = new PrintWriter(new File(nom));
            for (Autor a: autores.values()) {
                salida.println(a.getId()+" # "+a.getNombre());
            }
            salida.flush();
            salida.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public HashMap<String, Autor> getRepositorioAutores(){
        return autores;
    }
}

```

REPOSITORIO PUBLICACIONES

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class RepositorioPublicaciones {
    private HashMap<String, ArrayList<String>> citas; //ID Publicación # Publi
citada
    private HashMap<String, Publicacion> publicaciones; //ID Publicación #
Objeto Publicación
    private HashMap<String, ArrayList<String>> autores; //ID publicación #
Autores que la han realizado

    public RepositorioPublicaciones() {
        citas = new HashMap<>();
        publicaciones = new HashMap<>();
        autores = new HashMap<>();
    }

    public void readCitas(String nom) {
        try {
            Scanner entrada = new Scanner(new FileReader(nom));
            String linea;
            while(entrada.hasNextLine()) {
                linea = entrada.nextLine();
                String[] datos = linea.split(" # ");
                if (!citas.containsKey(datos[0])) {
                    citas.put(datos[0], new ArrayList<>());
                }
                citas.get(datos[0]).add(datos[1]);
            }
            entrada.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
}
```

```
public void readAutores(String nom) {  
    try {  
        Scanner entrada = new Scanner(new FileReader(nom));  
        String linea;  
        while(entrada.hasNextLine()) {  
            linea = entrada.nextLine();  
            String[] datos = linea.split(" # ");  
            if (!autores.containsKey(datos[0])) {  
                autores.put(datos[0], new ArrayList<>());  
            }  
            if(datos.length == 2) {  
                autores.get(datos[0]).add(datos[1]);  
            }  
        }  
        entrada.close();  
    }catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

```
public void readPublicaciones(String nom) {  
    try {  
        Scanner entrada = new Scanner(new FileReader(nom));  
        String linea;  
        while(entrada.hasNextLine()) {  
            linea = entrada.nextLine();  
            String[] datos = linea.split(" # ");  
            Publicacion publi = new Publicacion(datos[0], datos[1]);  
            publicaciones.put(datos[0], publi);  
        }  
        entrada.close();  
    }catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

```

public Publicacion buscarPubliPorId(String pId) {
    Publicacion publi = publicaciones.get(pId);
    return publi;
}

public void insertarPubli(String pId, String pTítulo) {
    Publicacion publi = new Publicacion(pId, pTítulo);
    publicaciones.put(pId, publi);
}

public void anadirCitaAPubli(String pId, String pCita) {
    if (!citas.containsKey(pId)) {
        citas.put(pId, new ArrayList<>());
    }
    citas.get(pId).add(pCita);
}

public void anadirAutorAPubli(String pIdPubli, String pIdAutor) {
    if (!autores.containsKey(pIdPubli)) {
        autores.put(pIdPubli, new ArrayList<>());
    }
    autores.get(pIdPubli).add(pIdAutor);
}

public void eliminarPubli(String pIdPubli) {
    publicaciones.remove(pIdPubli);
}

```

```

public ArrayList<Publicacion> listaPublisCitadas(String pId) {
    ArrayList<Publicacion> lPC = new ArrayList<>();
    ArrayList<String> lista = citas.get(pId);
    If(lista!=null) {

        for (String i: lista) {
            Publicacion publi = publicaciones.get(i);
            lPC.add(publi);
        }
    }
    return lPC;
}

public ArrayList<String> listaAutoresPubli(Publicacion p) {
    ArrayList<String> lA = autores.get(p.getId());
    return lA;
}

public ArrayList<Publicacion> listaPublicacionesAutor(Autor a){
    ArrayList<Publicacion> lista = new ArrayList<>();
    for(String pIdPubli: autores.keySet()) {
        if (autores.get(pIdPubli)!=null) {
            ArrayList<String> lA = autores.get(pIdPubli);
            if(lA.contains(a.getId())){
                lista.add(publicaciones.get(pIdPubli));
            }
        }
    }
    return lista;
}

```

```

public ArrayList<Publicacion> ordenarAlfabeticamente(){
    ArrayList<Publicacion> lista = new ArrayList<>();
    for(Publicacion publi: publicaciones.values()) {
        if(lista.size()==0) {
            lista.add(publi);
        } else {
            int posInicial=0, posFinal=lista.size()-1, posMitad=0;
            boolean enc=false;
            while(posInicial<=posFinal && !enc) {
                posMitad = (posInicial+posFinal)/2;
                Publicacion p = lista.get(posMitad);
                if(p.getId().compareTo(publi.getId())<0) {
                    posInicial = posMitad + 1;
                }else {
                    if(posMitad==0 || lista.get(posMitad - 1).getId().compareTo(publi.getId())<0) {
                        enc = true;
                    }else {
                        posFinal = posMitad - 1;
                    }
                }
            }
            if(!enc) {
                lista.add(publi);
            }else {
                lista.add(posMitad,publi);
            }
        }
    }
    return lista;
}

```

```

public void loadCitas(String nom) {
    try {
        PrintWriter salida = new PrintWriter(new File(nom));
        for (String pId: citas.keySet()) {
            for (String pCita: citas.get(pId)) {
                salida.println(pId+" # "+pCita);
            }
        }
        salida.flush();
        salida.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

```

public void loadAutores(String nom) {
    try {
        PrintWriter salida = new PrintWriter(new File(nom));
        for (String pId: autores.keySet()) {
            for (String pAutor: autores.get(pId)) {
                salida.println(pId+" # "+pAutor);
            }
        }
        salida.flush();
        salida.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

```

public void loadPublicaciones(String nom) {
    try {
        PrintWriter salida = new PrintWriter(new File(nom));
        for (String pId: publicaciones.keySet()) {
            salida.println(pId+"#" + publicaciones.get(pId).getTitulo());
        }
        salida.flush();
        salida.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

public HashMap<String, Publicacion> getPublicaciones(){
    return publicaciones;
}

public HashMap<String, ArrayList<String>> getCitas(){
    return citas;
}

public HashMap<String, ArrayList<String>> getAutores(){
    return autores;
}
}

```

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        System.out.println("TIEMPOS DE EJECUCIÓN");
        System.out.println(" ");
        System.out.println("Repositorio Autores:");

        //Leer Autores
        double millisInicio = System.currentTimeMillis();
        RepositorioAutores ra = new RepositorioAutores();
        ra.readAutores("Datuak/Datuak/authors-name-all.txt");
        double millisFin = System.currentTimeMillis();
        System.out.println("ReadAutores:  "+((millisFin-millisInicio)/1000)+"
segundos");

        //Load Autores
        millisInicio = System.currentTimeMillis();
        ra.loadAutores("Datuak/Datuak/JUnit loadAutores 2.txt");
        millisFin = System.currentTimeMillis();
        System.out.println("LoadAutores:  "+((millisFin-millisInicio)/1000)+"
segundos");

        System.out.println(" ");
        System.out.println("Repositorio Publicaciones");

        //Leer Citas
        millisInicio = System.currentTimeMillis();
        RepositorioPublicaciones rp = new RepositorioPublicaciones();
        rp.readCitas("Datuak/Datuak/publications-citedPubs-all.txt");
        millisFin = System.currentTimeMillis();
        System.out.println("ReadCitas:"+((millisFin-millisInicio)/1000)+"
segundos");

        //Leer Autores de Publicaciones
        millisInicio = System.currentTimeMillis();
        rp.readAutores("Datuak/Datuak/publications-authors-all-final.txt");
        millisFin = System.currentTimeMillis();
        System.out.println("ReadAutoresPublis"+((millisFin-
millisInicio)/1000)+" segundos");
    }
}
```

```

//Leer Publicaciones
milisInicio = System.currentTimeMillis();
rp.readPublicaciones("Datuak/Datuak/publications-titles-all.txt");
milisFin = System.currentTimeMillis();
System.out.println("ReadPublicaciones:"+(milisFinmilisInicio)/100)
+" segundos");

//Buscar Publi Por ID
milisInicio = System.currentTimeMillis();
rp.buscarPubliPorId("Q33205611");
milisFin = System.currentTimeMillis();
System.out.println("BuscarPubliPorID:"+(milisFin-
milisInicio)/1000)+" segundos");

//ListaPublisCitadas
milisInicio = System.currentTimeMillis();
rp.listaPublisCitadas("Q21136163");
milisFin = System.currentTimeMillis();
System.out.println("ListaPublisCitadas:"+(milisFinmilisInicio)/100
0)+" segundos");

//ListaAutoresPubli
milisInicio = System.currentTimeMillis();
rp.listaAutoresPubli(rp.buscarPubliPorId("Q101088249"));
milisFin = System.currentTimeMillis();
System.out.println("ListaAutoresPubli:"+(milisFinmilisInicio)/100)
+" segundos");

//ListaPublicacionesAutor
milisInicio = System.currentTimeMillis();
Autor a1 = new Autor("Q47372720", "Andrew J. Stewart");
rp.listaPublicacionesAutor(a1);
milisFin = System.currentTimeMillis();
System.out.println("ListaPublicacionesAutor:"+(milisFin-
milisInicio)/1000)+" segundos");

```

```

//OrdenarAlfabeticamente
milisInicio = System.currentTimeMillis();
rp.ordenarAlfabeticamente();
milisFin = System.currentTimeMillis();
System.out.println("OrdenarAlfabeticamente:"+(milisFin-
milisInicio)/1000)+" segundos");

//Load Autores
milisInicio = System.currentTimeMillis();
rp.loadAutores("Datuak/Datuak/JUnit loadAutores.txt");
milisFin = System.currentTimeMillis();
System.out.println("LoadAutores:  "+(milisFin-milisInicio)/1000)+"
segundos");

//Load Publicaciones
milisInicio = System.currentTimeMillis();
rp.loadPublicaciones("Datuak/Datuak/JUnit loadPublis.txt");
milisFin = System.currentTimeMillis();
System.out.println("LoadPublicaciones:"+(milisFin-
milisInicio)/1000)+" segundos");

//Load Citas
milisInicio = System.currentTimeMillis();
rp.loadCitas("Datuak/Datuak/JUnit loadCitas.txt");
milisFin = System.currentTimeMillis();
System.out.println("LoadCitas:"+(milisFin-milisInicio)/1000)+"
segundos");

```

```

}

```

```

}

```

6 Conclusiones

La solución desarrollada nos ha permitido gestionar de manera sencilla y rápida las publicaciones, autores y citas. El uso de estructuras como HashMap y ArrayList ha resultado muy útil para relacionar los datos y acceder a ellos directamente a partir de un identificador, lo que hace que las operaciones sean más rápidas y eficientes.

Entre los resultados principales, hemos conseguido implementar un sistema que carga correctamente la información desde los ficheros, permite añadir y eliminar elementos, así como consultar las relaciones entre publicaciones, autores y citas de forma ágil. Y, además, guardar la información actualizada y modificada en los ficheros.

Las características principales de la solución son su sencillez, la rapidez de acceso a los datos, la facilidad para escalar con nuevas funcionalidades y evitar la redundancia de datos, ya que al haber hecho las relaciones entre clases mediante HashMap, no se repiten ni estructuras, ni métodos, ni atributos.

La principal dificultad encontrada ha sido aprender a manejar los HashMap, ya que al principio es complejo entender cómo guardar y acceder a la información, pero con la práctica se ha visto que es una forma mucho más simple y cómoda de trabajar con este tipo de datos, puesto que ahorra hacer métodos del tipo recorrer listas para encontrar elementos, posiciones... Ya que dichos HashMap tienen métodos como get(), put(), containsKey()... con costes constantes $O(1)$.

Como posible línea futura de trabajo, se podría mejorar la solución añadiendo más validaciones de datos, o incluso implementar una interfaz que permita visualizar y manejar la información de una manera más intuitiva.